

Coordinación de subagentes: tokens, tiempo y cumplimiento

Cristhian Egoavil / 13 de septiembre de 2026 / v1.0

Resumen

Se revisan nueve papers de arXiv publicados inicialmente en 2026 y se comparan gestión secuencial, paralela, jerárquica y reflexiva, con un agente único de referencia y una variante de carga de skills. Doce tareas sintéticas, dos repeticiones y seis configuraciones producen 144 ejecuciones reales y 416 llamadas a GPT-5.6 Luna con pensamiento bajo. El agente único completa 20/24 tareas estrictas con 1 143 tokens y 4.72 segundos medios; las cuatro configuraciones de equipo completan 19–20/24. Una sensibilidad posterior al envoltorio SQL cambia los aciertos totales de 120/144 a 142/144 sin alterar las consultas. Cargar todos los skills aumenta los tokens medios jerárquicos un 94.8 %. Son resultados exploratorios sobre tareas cortas con datos completos; no establecen una arquitectura universalmente mejor ni autonomía prolongada.

Palabras clave: sistemas multiagente; orquestación; skills; eficiencia de tokens; evaluación reproducible.

1. Introducción

Un asistente personal puede delegar trabajo en agentes especializados, pero cada delegación consume recursos antes de producir trabajo útil. Hay que preparar instrucciones, transferir contexto, esperar respuestas e integrar resultados. Un equipo puede terminar antes un trabajo independiente; también puede repetir varias veces un cálculo sencillo sin mejorar la respuesta. La pregunta de ingeniería es, por tanto, cuándo la coordinación justifica su coste.

Este estudio investiga cómo elegir una arquitectura de subagentes considerando cumplimiento de tareas, consumo de tokens y tiempo transcurrido. Compara cuatro patrones acotados de coordinación con un solucionador individual y evalúa por separado cargar una biblioteca completa de skills. La contribución es una comparación empírica pequeña y transparente: tareas fijas, llamadas reales, evaluación determinista, registros de recursos por llamada y materiales editables bilingües. No introduce un modelo entrenado ni establece un benchmark de arquitectura en producción.

Se distinguen tres cantidades que a menudo se confunden. El éxito describe el resultado entregado. Los tokens totales miden el trabajo procesado entre todas las llamadas, incluido el contexto repetido. El tiempo transcurrido mide la espera del usuario y permite que las llamadas se solapen. Un contexto más corto en un trabajador no implica, por sí solo, menos tokens totales ni menor uso de RAM en la aplicación.

2. Investigación publicada en 2026

La revisión incluye nueve papers enviados inicialmente a arXiv entre febrero y septiembre de 2026. Se comprobaron las fechas de envío por separado de las revisiones; las versiones consultadas figuran

en las referencias. La selección se centró en coordinación, traspaso de contexto, asignación de skills y eficiencia medible. Es una revisión técnica focalizada, no una revisión sistemática exhaustiva. Sus puntuaciones no se combinan con el experimento posterior.

2.1. Topología de ejecución y autoridad

Kulkarni y Kulkarni comparan flujos secuenciales, paralelos, supervisor–trabajador y reflexivos para extraer documentos financieros. Sus descripciones motivan las cuatro familias estudiadas aquí. El dominio y la configuración de modelos/precios de enero de 2025 limitan la extrapolación a un asistente personal; no se reproducen sus configuraciones ni su rendimiento publicado.¹

DESBench estudia coordinación centralizada, jerárquica, heterárquica y holónica en planificación industrial. Su heterarquía implementada utiliza pujas mediadas. Amplía la pregunta desde la forma del grafo hasta quién puede asignar trabajo, rechazar compromisos y resolver conflictos; su simulador industrial constituye otro entorno de evaluación.²

ParaGUI envía trabajadores de interfaz gráfica en paralelo por rondas e integra sus resúmenes. Aporta evidencia sobre trabajo interactivo divisible e identifica la ejecución del trabajador como una limitación. Sus controles del historial visual y el entorno de varios escritorios impiden trasladar sus cifras directamente a este experimento textual.³

2.2. Traspaso de información y evaluación

OrchBench representa dependencias y transferencias entre agentes y simula calidad, tiempo de terminación y tokens. Su mecanismo de retención convierte la pérdida de información en una variable de coordinación. Aquí se ejecutan todos los trabajadores contra la API; no se sustituyen consumo o respuestas observados por resultados o costes simulados.⁴

CRAFT entrega vistas parciales complementarias de una construcción. Distingue razonamiento individual y comunicación efectiva y describe correcciones repetidas improductivas. Motiva conservar evidencia y evaluar el resultado final. Como todos nuestros trabajadores reciben la tarea original, no se estudia su problema de observación parcial.⁵

2.3. Skills y contexto específico

Subagents vs Agent Skills distingue paquetes procedurales con interfaces explícitas de entrada/salida y texto poco estructurado. Su evaluación principal utiliza un subconjunto seleccionado de 64 tareas e informa menor contexto máximo con mayor consumo total. Esto respalda estudiar la carga de información por separado de la topología.⁶

EvoAgent combina delegación jerárquica y selección por palabras clave, embeddings y modelo. Su evaluación de dominio utiliza jueces modelo; esas puntuaciones no establecen fiabilidad de ejecución exacta. Se toma como referencia la separación de recuperación y delegación, sin implementar aprendizaje autónomo ni reproducir su cascada de selección.⁷

¹Kulkarni y Kulkarni, arXiv:2603.22651v1, 2026.

²Wang et al., arXiv:2605.13172v1, 2026.

³Yu et al., arXiv:2607.22689v1, 2026.

⁴Ren et al., arXiv:2607.25656v1, 2026.

⁵Nath, VanderHoeven y Krishnaswamy, arXiv:2603.25268v2, 2026.

⁶Piriyakulkij et al., arXiv:2609.09233v1, 2026.

⁷A. Zhang et al., arXiv:2604.20133v3, 2026.

Swarm Skills empaqueta roles, flujos, límites y dependencias como recursos reutilizables de coordinación. Su caso cualitativo deja pendiente validar ampliamente la compatibilidad. Se consideran las descripciones portables una propuesta de diseño, no evidencia de que cualquier framework multiagente pueda intercambiarlas sin comprobar su funcionamiento.⁸

Xu y Yan revisan la carga progresiva: descubrir metadatos, cargar instrucciones elegidas y recuperar recursos de apoyo. Esto distingue el catálogo del contenido insertado en cada contexto. Como revisión, orienta la política de carga, pero no aporta un resultado de rendimiento independiente para nuestras arquitecturas.⁹

3. Arquitecturas implementadas

Todos los nodos llaman al mismo modelo mediante solicitudes independientes sin estado. Un rol se implementa con instrucciones y un traspaso explícito; no es otro especialista entrenado ni un proceso persistente. Las cuatro configuraciones multiagente cambian el flujo de información y decisiones. Son implementaciones pequeñas de familias de patrones, no réplicas de los frameworks citados.

Configuración	Flujo de control	Llamadas
Único	Un solucionador recibe la petición completa.	1
Secuencial	Extraer evidencia; resolver; integrar, con el último traspaso.	3
Paralela	Dos especialistas complementarios fijos en paralelo; integración posterior.	3
Jerárquica	Supervisor elige 1–2 encargos; trabajadores ejecutan; supervisor integra.	3–4
Reflexiva	Proponer; criticar; revisar si se rechaza. Máximo dos rondas de crítica/revisión.	2–5

Cuadro 1: Comparación principal. La configuración adicional con todos los skills conserva el flujo jerárquico. Una llamada es una solicitud completa a la API.

Secuencial. El primer trabajador extrae hechos y restricciones. El segundo calcula un candidato y el nodo final lo contrasta con la petición original. Cada etapa recibe el original y solo el último traspaso, evitando un historial ilimitado. Ofrece un orden predecible, pero sitúa las tres llamadas en la ruta que determina la espera.

Paralela. Un trabajador aborda datos y cálculos principales; el otro, excepciones, ámbito y dependencias. Ambos reciben la tarea completa. Los encargos fijos evitan una llamada de planificación y el integrador ve ambas salidas. Se evalúa análisis complementario en paralelo, no una votación ni acciones externas simultáneas.

Jerárquica. La primera llamada produce una lista JSON con uno o dos encargos concretos. Los trabajadores reciben su encargo y la tarea original; la llamada final recibe plan y hallazgos. Un plan inválido cuenta como fallo. No se permiten subagentes de segundo nivel y siempre se delega: la opción de resolver directamente corresponde a la referencia individual.

Reflexiva. Un proponente entrega un candidato. Un crítico independiente lo aprueba sin observaciones o indica correcciones. La revisión recibe candidato y crítica. Se permiten dos rondas como máximo y se evalúa el último candidato aunque la última revisión autorizada no tenga aprobación

⁸X. Zhang et al., arXiv:2605.10052v2, 2026.

⁹Xu y Yan, arXiv:2602.12430v4, 2026.

posterior. El crítico nunca ve respuestas de referencia ni resultados de pruebas.

Las configuraciones principales reciben los metadatos de diez skills y un skill procedural pertinente. Una variante jerárquica carga los diez cuerpos completos en cada nodo. La selección utiliza la etiqueta conocida del tipo de tarea, no una llamada de inferencia. Se aísla la política de carga; no se demuestra que el asistente pueda seleccionar siempre el skill correcto desde lenguaje natural.

4. Método experimental

4.1. Tareas y evaluación objetiva

La evaluación incluye doce instancias distintas diseñadas para el estudio. Son sintéticas y autosuficientes: se entregan datos, esquema y reglas, mientras las respuestas esperadas y bases de prueba permanecen en el evaluador. El modelo evaluado no dispone de web, modificaciones de archivos ni terminal. SQL solo se ejecuta en bases temporales en memoria con autorización de solo lectura y límite de trabajo por sentencia.

IDs	Grupo	Requisitos evaluados
T01–02	Registros	Resolver revisiones antes de filtrar; sumar céntimos enteros con signo por proyecto.
T03–04	Calendario	Primer intervalo común para tres personas, desfases UTC fijos e intervalos ocupados semiabierto.
T05–06	Dependencias	Inicios mínimos y ruta crítica con desempate lexicográfico en once nodos de un DAG.
T07–08	Selección	Elegir cuatro de doce elementos con presupuesto, cobertura, pre-requisitos e incompatibilidades.
T09–10	Memoria	Reducir 24 eventos por ámbito, conservando procedencia, preferencias locales y candidatos sin confirmar.
T11–12	SQL	Sumar sin multiplicar uniones; elegir últimos intentos con desempates; conservar entidades vacías.

Cuadro 2: Dos instancias por grupo. Cada instancia SQL tiene tres bases distintas; superar las tres cuenta como una tarea completada.

Diez tareas exigen una respuesta estructurada exacta. La referencia utiliza reducción de eventos, búsqueda de intervalos, programación dinámica o enumeración exhaustiva. Las dos tareas SQL requieren una consulta que supere las tres bases. El formato principal es un objeto que contiene únicamente **answer**, cuyo valor debe respetar la estructura pedida. Una forma incorrecta falla aunque una persona pueda interpretarla. Se conserva el porcentaje de bases SQL superadas como diagnóstico secundario.

Siete pruebas locales del arnés abarcan referencias válidas, respuestas plausibles incorrectas, multiplicación de uniones y desempates SQL, solo lectura, concurrencia, respuestas incompletas y conteo de tokens. Validan la medición y no se cuentan como aciertos del agente. Un piloto separado de seis configuraciones utilizó otra instancia contable; se conserva su evidencia y se excluye de las tablas de rendimiento.

4.2. Controles de ejecución

Cada tarea se ejecuta dos veces en seis configuraciones: 144 ejecuciones, 120 de la comparación principal y 24 de carga de skills. Los bloques tarea/repetición y el orden interno de configuraciones

se aleatorizan con semilla 20260913. Solo hay una ejecución a la vez y hasta dos llamadas de trabajadores simultáneas dentro de los flujos compatibles. El protocolo local y los hashes se fijaron antes de evaluar; no hubo prerregistro externo.

Todas las solicitudes usan `gpt-5.6-luna`, pensamiento `low`, API Responses, modo objeto JSON, sin streaming y `store:false`. El máximo por llamada es 1.536 tokens de salida, incluido razonamiento. No se definen temperatura ni semilla del modelo. La documentación oficial confirma identificador y esfuerzo; es orientación de implementación fuera del corpus de nueve papers.¹⁰

El techo común es de cinco llamadas. No comienza otra después de 240 segundos; los límites HTTP de conexión/lectura son 10/60 segundos. Es un plazo de inicio, no una interrupción estricta del tiempo total. No hay reintentos de transporte ni caché propia de respuestas. La caché de entrada del proveedor sigue siendo posible y se registra. Se conservan los fallos sin repeticiones selectivas. No se ajustaron prompts, tareas, límites ni evaluador principal según el rendimiento observado.

Se igualan modelo, acceso a datos y máxima salida por llamada, no el cómputo consumido. Un solucionador de una llamada y un flujo de varias gastan cantidades distintas. Se comparan, por tanto, configuraciones completas de roles y orquestación; no se puede aislar la topología del efecto de más inferencia o del texto de cada rol.

4.3. Métricas e incertidumbre

Para la configuración a , s_{ai} indica éxito completo, t_{ai} suma tokens de entrada y salida informados y w_{ai} es tiempo transcurrido. Con $N = 24$ ejecuciones por configuración, las medidas principales son

$$S_a = \frac{1}{N} \sum_i s_{ai}, \quad \bar{T}_a = \frac{1}{N} \sum_i t_{ai}, \quad \bar{W}_a = \frac{1}{N} \sum_i w_{ai}. \quad (1)$$

La eficiencia es $10^5 \sum_i s_{ai} / \sum_i t_{ai}$ aciertos por 100.000 tokens y $60 \sum_i s_{ai} / \sum_i w_{ai}$ aciertos por minuto acumulado. Ambas incluyen intentos fallidos. La última describe una carga secuencial, no la capacidad de un servidor saturado.

Se cuentan supervisor, crítico, entrada repetida y razonamiento. La entrada cacheada y el razonamiento ya están incluidos en el total y no se suman otra vez. El tiempo incluye orquestación, espera de red y escritura de trazas, pero excluye el evaluador final. Se guarda también la suma de duraciones API, que no se usa como latencia cuando hay solapamiento. El consumo ausente convertiría los totales en cotas inferiores, sin reemplazarlos por estimaciones.

Se utilizan 10.000 muestras bootstrap pareadas por tarea con semilla 20260913: se remuestran doce identificadores conservando sus dos repeticiones y todas las configuraciones. Los percentiles delimitan intervalos descriptivos del 95 % para éxito, tokens medios y tiempo medio; la comparación de skills usa diferencias pareadas. La unidad independiente es la instancia, no cada repetición. Son intervalos exploratorios de una muestra pequeña; la distribución sintética no representa todo el trabajo de un asistente.

5. Resultados

Terminaron las 144 ejecuciones previstas. La auditoría reconcilia 416 llamadas y 568 283 tokens informados por el proveedor, incluidas las 24 ejecuciones de carga de skills. Hubo 0 errores de infraestructura o ejecución del protocolo; todas las ejecuciones tienen consumo completo. Se

¹⁰OpenAI, documentación GPT-5.6 Luna, consultada el 13 de septiembre de 2026.

excluyen las seis del piloto. Los hashes iniciales y finales coinciden y el orden registrado corresponde al plan fijado.

Se evaluó el 13 de septiembre de 2026 entre 13:09:45 y 13:33:34 UTC. Son tiempos observados ese día contra la API, no una previsión para otras cuentas o despliegues.

Configuración	Estricto	SQL norm.	Tokens/ej.	Media s	Mediana s
Agente único	20/24	23/24	1 143	4.72	3.39
Secuencial	20/24	24/24	3 410	13.72	11.77
Paralela	19/24	23/24	3 520	8.14	6.04
Jerárquica	20/24	24/24	4 373	11.19	10.01
Reflexiva	20/24	24/24	2 714	9.77	6.26
Jerárquica + todos los skills	21/24	24/24	8 518	11.93	10.88

Cuadro 3: Resultados observados. Estricto es el criterio principal fijado. SQL norm. es la sensibilidad posterior al envoltorio; no cambia el texto de la consulta. La última fila separada corresponde a la carga de skills.

5.1. Cumplimiento de formato y calidad de la solución

Durante la evaluación, las respuestas SQL mostraron una diferencia entre una consulta aparentemente correcta y la estructura anidada solicitada. Se conservó el evaluador principal. Una sensibilidad explícitamente posterior envuelve una cadena existente dentro de answer en un objeto con campo sql y ejecuta la misma consulta contra las tres bases. No se corrige ni repite texto SQL, respuesta numérica, prompt o intento.

La normalización se aplica a 22 salidas registradas y cambia los aciertos completos de 120/144 a 142/144 entre las seis configuraciones. La columna estricta mide también cumplimiento del formato entregado; la normalizada diagnostica cuánto fallo se atribuye al envoltorio. No es otro objetivo prefijado ni prueba de que todas las consultas funcionen fuera de estas bases.

Los dos fallos restantes corresponden a T05, repetición 2, en las configuraciones individual y paralela. Ambas entregan la duración total correcta de 38, pero inician H en 10 en vez de 18 y K en 21 en vez de 29. H depende de que termine C y K de que termine H. Es un error al propagar dependencias, no de envoltorio JSON; comprobar solo la duración final lo ocultaría.

5.2. Skills seleccionados frente al catálogo completo

La configuración jerárquica consume 4 373 tokens medios con un skill seleccionado y 8 518 al cargar los diez cuerpos. La diferencia media pareada es 4 145 tokens, con intervalo del 95 % [3898; 4366]. El incremento es 94.8 %, intervalo [87.9; 101.9] %.

El éxito estricto pasa de 20/24 a 21/24; con la sensibilidad de envoltorio pasa de 24/24 a 24/24. La diferencia media de latencia, todos menos seleccionado, es 0.74 segundos, intervalo [-0.24; 1.83]. Es una comparación de carga con el grupo correcto conocido de antemano; no mide errores de búsqueda de skills ni su coste.

5.3. Costes e incertidumbre

El flujo reflexivo ejecuta 5 llamadas de revisión en 24 ejecuciones; un crítico puede añadir coste aunque no haya revisión posterior. La API informa 44 788 tokens de entrada cacheada y 97 915



Figura 1: Éxito principal, tokens totales y tiempo transcurrido medidos. Las líneas muestran intervalos descriptivos del 95 % por tarea. La carga de todos los skills se distingue del resto. Figura generada con los datos entregados.

de razonamiento. Son subconjuntos, no tokens adicionales. Las tablas 4 y 5 conservan intentos fallidos y reflejan el pequeño número de instancias independientes. Un orden puntual no establece superioridad general estadística.

Configuración	Tokens totales	Llamadas	Aciertos/100k tok.	Aciertos/min
Agente único	27 424	24	72.93	10.58
Secuencial	81 845	72	24.44	3.64
Paralela	84 479	72	22.49	5.83
Jerárquica	104 963	95	19.05	4.47
Reflexiva	65 133	58	30.71	5.12
Jerárquica + todos los skills	204 439	95	10.27	4.40

Cuadro 4: Eficiencia con aciertos estrictos y todos los intentos. Aciertos por minuto acumulado no equivale a capacidad de un servidor saturado.

6. Cómo elegir una arquitectura de subagentes

En estas tareas cortas, la referencia individual consume 1 143 tokens medios, frente a 2 714 de la configuración de equipo que menos tokens utiliza. Sus aciertos estrictos deben interpretarse junto a la tabla 3 y la sensibilidad SQL. La recomendación práctica es conservar ejecución directa y evaluar delegación cuando el trabajo paralelo o separar contexto puedan compensar las llamadas adicionales. Es una recomendación para esta carga, no un rechazo de los subagentes.

6.1. Un agente maestro con delegación opcional

Un coordinador adaptativo puede elegir entre ejecución directa y delegación. El maestro interpreta la petición, mantiene el objetivo activo e integra los resultados. Delegar es una operación disponible,

Registros	4/4	4/4	4/4	4/4	4/4	4/4
Calendario	4/4	4/4	4/4	4/4	4/4	4/4
Dependencias	3/4	4/4	3/4	4/4	4/4	4/4
Selección	4/4	4/4	4/4	4/4	4/4	4/4
Memoria	4/4	4/4	4/4	4/4	4/4	4/4
SQL	1/4	0/4	0/4	0/4	0/4	1/4
	Agente único	Secuencial	Paralela	Jerárquica	Reflexiva	Jerárquica (todos los skills)

Aciertos exactos por grupo: dos instancias x dos repeticiones

Figura 2: Aciertos estrictos por grupo. Cada celda corresponde a cuatro ejecuciones, no cuatro instancias independientes. La sensibilidad SQL figura por separado en la tabla 3.

Configuración	Éxito %	Tokens medios	Segundos medios
Agente único	[62.5; 100.0]	[1005; 1279]	[3.33; 6.43]
Secuencial	[58.3; 100.0]	[3094; 3752]	[10.90; 17.28]
Paralela	[54.2; 100.0]	[3157; 3904]	[5.65; 11.58]
Jerárquica	[58.3; 100.0]	[4012; 4740]	[9.04; 13.89]
Reflexiva	[58.3; 100.0]	[2373; 3064]	[7.03; 12.82]
Jerárquica + todos los skills	[66.7; 100.0]	[7987; 9037]	[9.39; 15.22]

Cuadro 5: Intervalos descriptivos del 95 % por bootstrap de tareas para éxito principal, tokens medios y tiempo medio. Se remuestran doce identificadores de tarea.

no una etapa obligatoria para cada mensaje. Esta arquitectura condicional es una recomendación derivada de los intercambios observados; su política de enrutamiento no se entrenó ni evaluó aquí.

Para dos trabajadores en paralelo, una aproximación de la latencia es

$$W_{\text{equipo}} \approx W_{\text{plan}} + \max(W_1, W_2) + W_{\text{integracion}}, \quad (2)$$

mientras los tokens se suman entre planificador, ambos trabajadores e integrador. La concurrencia solo acorta la parte solapada de la ruta que determina la espera. Una decisión útil debe considerar trabajo separable, trasposos necesarios y duración esperada, además de la existencia de subtareas. Una acción directa pequeña puede no ofrecer trabajo suficiente para compensar la coordinación.

La primera implementación podría permitir un nivel de delegación y dos trabajadores simultáneos, con techo configurable. Son valores para un diseño inicial manejable, no un óptimo demostrado. Aumentar el equipo, incorporar negociación entre pares o mezclar tamaños de modelo requiere mediciones controladas adicionales.

6.2. Asignación de skills y contexto

El maestro debería descubrir el catálogo completo de metadatos y los trabajadores cargar inicialmente las instrucciones pertinentes a su encargo. Cada trabajador podría solicitar más recursos permitidos si el paquete inicial resulta insuficiente. Se mantiene la capacidad de descubrir funcionalidades sin copiar todas las instrucciones en cada petición. La selección por etiqueta del experimento es más sencilla que el enrutamiento real; su ahorro no garantiza el de un selector desplegado.

Un traspaso práctico puede incluir `task_id`, `parent_task_id`, objetivo, referencias de entrada, versiones de skills, herramientas permitidas, dependencias y forma del resultado. La respuesta debería separar trabajo completado, artefactos, hechos sin resolver y bloqueos. Estos campos organizan ejecución y auditoría; no requieren otra llamada al modelo para reformular una prueba formal por cada acción.

Los trabajadores experimentales recibieron el original completo para igualar acceso a evidencia. El contexto específico de producción sería un cambio posterior: suministrar evidencia de la tarea e instrucciones recientes relevantes, con referencias recuperables para lo omitido. Se debe medir tanto reducción de contexto como fallos por información ausente. Los resultados actuales no demuestran que un contexto menor siempre sea suficiente.

6.3. Responsabilidades del motor

Los trabajadores pueden programarse como tareas asíncronas que comparten clientes de proveedores y conexiones de almacenamiento. Un proceso es necesario cuando una herramienta lo requiere. Así se evita equiparar cada agente con un proceso completo del sistema operativo, pero este estudio no midió memoria del host ni afirma una reducción de RAM.

El planificador debe limitar concurrencia, propagar cancelación y conservar dependencias. Un maestro que espera hijos no debe ocupar el único puesto de ejecución que ellos necesitan; la admisión de coordinadores y trabajadores debe ser compatible. Los permisos de un trabajador no pueden superar los del padre y las escrituras en conflicto requieren coordinación.

El coordinador puede ofrecer delegación, estado, comunicación y cancelación mediante una interfaz pequeña del motor. Escalar las decisiones sin resolver al coordinador conserva un límite claro de

responsabilidad. Los resultados tipados de herramientas y el manejo ordinario de errores siguen siendo responsabilidades del motor. El crítico modelo estudiado en la configuración reflexiva es cómputo adicional opcional cuyo valor debe demostrarse para el trabajo concreto.

7. Límites, reproducibilidad y próximo experimento

Las tareas son pocas, sintéticas y diseñadas para este estudio. Las entradas breves y completas favorecen resolver directamente y no ejercitan recuperación de documentos, terminal, cambios de código en varios archivos ni recuperación prolongada ante fallos. Solo se utilizó un modelo y esfuerzo, con dos repeticiones por instancia. La caché del proveedor, la planificación de solicitudes y la red del día afectan la latencia. Igualar límites máximos no equivale a igualar cómputo consumido.

La sensibilidad posterior al formato SQL permite interpretar los errores principales, pero no es otro objetivo prerregistrado. Solo cambia el envoltorio de una cadena existente y conserva las puntuaciones estrictas originales. No corrige lógica SQL, contenido ni fallos de infraestructura. Su interpretación debe mantenerse separada del análisis principal fijado.

El paquete entregado incluye generadores, prompts exactos, diez skills, evaluadores, protocolo local, hashes, orden aleatorio, respuestas visibles, consumo, marcas temporales, tablas CSV/XLSX y ambas versiones LaTeX. La auditoría comprueba bloques completos y reconcilia tokens por llamada y ejecución. Un registro de enmiendas editoriales documenta los cambios de redacción de los protocolos posteriores a la evaluación, sin sustituir hashes históricos ni modificar la configuración experimental. Se excluyen credenciales y razonamiento privado. El alias del modelo puede evolucionar: se reproduce el método y análisis registrado, no un servicio remoto determinista.

El siguiente experimento debería reservar tareas reales con acciones externas, contextos extensos, dependencias explícitas y escrituras incompatibles; incorporar más modelos y comparar delegación adaptativa con ejecución directa a presupuestos equivalentes. También debería variar por separado ámbito del contexto, descubrimiento de skills y tamaño del equipo. Este estudio aporta una medición inicial reproducible, no evidencia de que un equipo fijo sea universalmente mejor.

8. Conclusión

La arquitectura debe elegirse considerando corrección entregada, inferencia total y espera real conjuntamente. La comparación hace visible el coste de coordinar y muestra por qué se debe distinguir cumplimiento de formato y calidad de la solución. Para las tareas breves medidas, la ejecución directa es la referencia más económica. Entre los patrones con subagentes, la coordinación reflexiva consume menos tokens en promedio y la paralela presenta menor latencia media. No se establece un ganador universal; la delegación adaptativa con contexto específico sigue siendo una hipótesis que debe evaluarse en distintas cargas de trabajo.

Disponibilidad y estado. El artículo bilingüe y el paquete experimental están disponibles en la página de investigación de CREH Labs. Es un preprint exploratorio sin revisión por pares. El benchmark se ejecutó en un arnés de investigación independiente y evalúa patrones acotados de coordinación.

Referencias

1. Siddhant Kulkarni y Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies*. 2603.22651v1. Primer envío: 2026-03-24. <https://arxiv.org/html/2603.22651v1>
2. Ziqi Wang, Yuhao Yang, Zhiwei Ling, Wenzhuo Qian y Hailiang Zhao. *When Does Hierarchy Help? Benchmarking Agent Coordination in Event-Driven Industrial Scheduling*. 2605.13172v1. Primer envío: 2026-05-13. <https://arxiv.org/html/2605.13172v1>
3. Zedong Yu et al.. *Beyond Sequential Interaction: Benchmarking Parallel Execution and Coordination for GUI Agents*. 2607.22689v1. Primer envío: 2026-07-17. <https://arxiv.org/html/2607.22689v1>
4. Zhenzhen Ren et al.. *OrchBench: Evaluating Multi-Agent Orchestration Plans in Isolation via Deterministic Simulation*. 2607.25656v1. Primer envío: 2026-07-28. <https://arxiv.org/html/2607.25656v1>
5. Abhijnan Nath, Hannah VanderHoeven y Nikhil Krishnaswamy. *CRAFT: Grounded Multi-Agent Coordination Under Partial Information*. 2603.25268v2. Primer envío: 2026-03-26. <https://arxiv.org/html/2603.25268v2>
6. Wasu Top Piriyaikulij, Rachel Lawrence, Alicia Curth, Sushrut Karmalkar y Niranjani Prasad. *Subagents vs Agent Skills: Executing Reusable Knowledge for Long-Horizon Agentic Tasks*. 2609.09233v1. Primer envío: 2026-09-07. <https://arxiv.org/html/2609.09233v1>
7. Aimin Zhang, Jiajing Guo, Fuwei Jia, Chen Lv, Boyu Wang y Fangzheng Li. *EvoAgent: An Evolvable Agent Framework with Skill Learning and Multi-Agent Delegation*. 2604.20133v3. Primer envío: 2026-04-22. <https://arxiv.org/html/2604.20133v3>
8. Xinyu Zhang et al.. *Swarm Skills: A Portable, Self-Evolving Multi-Agent System Specification for Coordination Engineering*. 2605.10052v2. Primer envío: 2026-05-11. <https://arxiv.org/html/2605.10052v2>
9. Renjun Xu y Yang Yan. *Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward*. 2602.12430v4. Primer envío: 2026-02-12. <https://arxiv.org/html/2602.12430v4>
10. OpenAI. *GPT-5.6 Luna Model*. Consultado el 2026-09-13. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>